Très bien Loïc — tu veux comprendre **en profondeur** ce qui se passe dans la partie *“Apache reçoit la requête”*.
C’est une excellente question, parce que cette étape est souvent négligée alors qu’elle est **cruciale** pour comprendre pourquoi une API REST fonctionne… ou casse.

Je vais t’expliquer ça **comme si on ouvrait le capot d’Apache** et qu’on suivait la requête en temps réel.

🔥 2. Quand Apache reçoit la requête : ce qui se passe réellement

Quand ton frontend Vue.js envoie :

```
POST http://localhost/api/public/tasks
```

Apache ne se contente pas de “recevoir une URL”.
Il déclenche **une chaîne de mécanismes internes** très précise.

Voici le déroulement complet.

🧩 Étape 2.1 — Apache reçoit la requête HTTP brute

Apache reçoit :

- la méthode : `POST`
- l’URL : `/api/public/tasks`
- les headers : `Content-Type: application/json`
- le body JSON : `{"title":"Nouvelle tâche","done":0}`

À ce moment-là, Apache ne sait **pas encore** si :

- c’est un fichier
- un dossier
- une route API
- un script PHP

- ou autre chose

Il ne fait que **stocker la requête en mémoire**.

🧩 Étape 2.2 — Apache cherche un fichier correspondant

Apache regarde dans le dossier configuré comme `DocumentRoot`.

Dans ton cas :

```
C:/wamp64/www/api/public
```

Il cherche :

- un fichier `tasks`
- un dossier `tasks`
- un fichier `tasks.php`
- un fichier `tasks.html`
- etc.

➡️ Il ne trouve **rien**.

Donc Apache se dit :
"OK, ce n'est pas un fichier. Je dois voir si un .htaccess me dit quoi faire."

🧩 Étape 2.3 — Apache charge le `.htaccess`

Apache lit ton fichier :

```
 RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [QSA,L]
```

Et il applique les règles **dans l'ordre**.

🧩 Étape 2.4 — Apache teste les conditions Rewrite

Condition 1 :

```
RewriteCond %{REQUEST_FILENAME} !-f
```

Apache vérifie :

“Est-ce que `/tasks` est un fichier ?”

Réponse : **non**
→ Condition validée.

Condition 2 :

```
RewriteCond %{REQUEST_FILENAME} !-d
```

Apache vérifie :

“Est-ce que `/tasks` est un dossier ?”

Réponse : **non**
→ Condition validée.

🧩 Étape 2.5 — Apache applique la règle Rewrite

Comme les deux conditions sont vraies, Apache applique :

```
RewriteRule ^ index.php [QSA,L]
```

Ce qui signifie :

“Peu importe l’URL, envoie tout vers index.php,
mais garde l’URL originale en mémoire.”

Donc Apache transforme :

```
/tasks
```

en :

```
/index.php
```

Mais il **n'efface pas** l'URL `/tasks`.
Il la stocke dans `$_SERVER['REQUEST_URI']`.

🧩 Étape 2.6 — Apache passe la main à PHP

Apache voit que `index.php` existe.
Il lance le moteur PHP et lui dit :

"Exécute index.php, voici la requête complète."

PHP reçoit :

- `$_SERVER['REQUEST_METHOD'] = "POST"`
- `$_SERVER['REQUEST_URI'] = "/api/public/tasks"`
- le body JSON brut dans `php://input`

🧩 Étape 2.7 — PHP commence à router la requête

Ton fichier `index.php` lit :

```
$uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
```

Donc PHP récupère **l'URL originale**, pas celle réécrite.

C'est grâce à ça que ton API REST fonctionne.

🧩 Étape 2.8 — Apache attend la réponse PHP

PHP exécute ton contrôleur, fait l'INSERT MySQL, renvoie un JSON.

Apache reçoit la réponse PHP :

```
{
  "id": 5,
  "title": "Nouvelle tâche",
  "done": 0
}
```

Et Apache renvoie ça au navigateur.

🎉 Résumé clair de la partie Apache

Étape	Ce que fait Apache
1	Reçoit la requête HTTP
2	Cherche un fichier correspondant
3	Charge `.htaccess`
4	Vérifie les conditions Rewrite
5	Réécrit l'URL vers `index.php`
6	Passe la requête à PHP
7	Attend la réponse
8	Renvoie la réponse JSON au frontend

💡 Pourquoi cette étape est essentielle ?

Parce que **sans Apache + .htaccess**, ton API REST ne fonctionnerait pas :

- `/tasks` n'existe pas physiquement
- `/tasks/1` non plus
- `/tasks/1/edit` encore moins

C'est Apache qui transforme **toutes** ces URL en :

```
index.php
```

et laisse ton routeur PHP faire le reste.

Si tu veux, je peux maintenant t'expliquer :

- comment Apache gère les erreurs 404 / 500
- comment fonctionne `mod_rewrite` en profondeur
- comment créer un VirtualHost propre pour ton API

Tu veux qu'on continue dans quelle direction ?